



Condicionales complejos y validación de inputs

En sistemas reales (y en sistemas con IA), **los inputs no son fiables**:

- el usuario se equivoca
- llegan valores vacíos o con espacios
- llegan tipos inesperados (string cuando esperas int)
- llegan formatos mal escritos (email, ID, código...)

La regla práctica es simple:

Validar antes de usar.

Objetivos

- Usar **and** / **or** / **not** para condiciones compuestas.
 - Validar texto (vacío, longitud) y números (tipo, rango).
 - Validar opciones con **whitelist** (**valid_options**).
 - Usar regex **básicas** con **re.fullmatch** para formatos simples.
 - Devolver errores claros (no solo **print("error")**).
-

1) Condicionales complejos

1.1 **and**, **or**, **not**

- **and**: todas las condiciones deben cumplirse.
- **or**: basta con que se cumpla una.
- **not**: niega una condición.

Ejemplo:

- "prioridad válida" si es entero **y** está entre 1 y 5:

```
if isinstance(prioridad, int) and 1 <= prioridad <= 5:
```

...

1.2 Comparaciones encadenadas

En Python puedes escribir:

```
1 <= prioridad <= 5
```

Es equivalente a:

```
prioridad >= 1 and prioridad <= 5
```

1.3 Evitar anidar demasiado

En validación suele ser más legible:

- comprobar
- si falla → devolver error
- si pasa → seguir

2) Validación con Python nativo

2.1 Normalizar: `strip()` y `lower()`

Si aceptas texto del usuario, normalmente conviene:

```
texto = texto.strip()
```

Si es una opción textual:

```
tipo = tipo.strip().lower()
```

2.2 Texto vacío y longitud mínima/máxima

```
if not texto.strip():  
    return False, "El texto no puede estar vacío."  
  
if len(texto.strip()) < 5:  
    return False, "El texto debe tener al menos 5 caracteres."
```

2.3 Tipos de datos y rango numérico (try/except)

Si el valor puede venir como string ("3"), convierte y controla errores:

```
try:
    n = int(valor)
except ValueError:
    return False, "Debe ser un número entero."

if not (1 <= n <= 5):
    return False, "Debe estar entre 1 y 5."
```

2.4 Opciones válidas (whitelist)

En sistemas mantenibles, define las opciones permitidas y valida contra ellas:

```
VALID_TIPOS = ["summary", "translate", "qa"]

if tipo not in VALID_TIPOS:
    return False, f"Tipo inválido. Usa una de: {VALID_TIPOS}"
```

2.5 Ejemplos - validaciones con Python nativo

Estos ejemplos son **didácticos**: no pretenden cubrir todos los casos "de producción", pero sí enseñar validación robusta con métodos de Python.

Ejemplo A — Validar un email

Reglas simples:

- exactamente un @
- usuario y dominio no vacíos
- dominio contiene al menos un .
- sin espacios

```
def validar_email_sin_regex(email: str) -> tuple[bool, str | None]:
    if not isinstance(email, str):
        return False, "Email inválido: debe ser un string."

    e = email.strip()
    if not e:
        return False, "Email inválido: no puede estar vacío."

    if " " in e:
        return False, "Email inválido: no debe contener espacios."
```

```

if e.count("@") != 1:
    return False, "Email inválido: debe contener un único '@'."

usuario, dominio = e.split("@")
if not usuario or not dominio:
    return False, "Email inválido: usuario y dominio son obligatorios."

if "." not in dominio:
    return False, "Email inválido: el dominio debe contener un punto (.)."

return True, None

```

Ejemplo B — Pedir una edad por `input()` y validarla

Reglas simples:

- debe ser un entero
- rango permitido (por ejemplo 0–120)

```

def pedir_edad(minimo: int = 0, maximo: int = 120) -> int:
    while True:
        raw = input("Edad: ").strip()

        if not raw:
            print("La edad no puede estar vacía.")
            continue

        # Opcional: permitir "+18" o "18"
        if raw.startswith("+"):
            raw = raw[1:]

        if not raw.isdigit():
            print("La edad debe ser un número entero (solo dígitos).")
            continue

        edad = int(raw)
        if not (minimo <= edad <= maximo):
            print(f"La edad debe estar entre {minimo} y {maximo}.")
            continue

        return edad

```

Ejemplo C — Validar una fecha recibida

Para fechas, lo más robusto es delegar el "formato" y el calendario a `datetime`.

```

from datetime import datetime

def validar_fecha(fecha: str, formato: str = "%Y-%m-%d") -> tuple[bool, str |

```

```
None]:
    if not isinstance(fecha, str):
        return False, "Fecha inválida: debe ser un string."

    f = fecha.strip()
    if not f:
        return False, "Fecha inválida: no puede estar vacía."

    try:
        datetime.strptime(f, formato)
    except ValueError:
        return False, f"Fecha inválida: formato esperado {formato} (ej. 2026-05-28)."

    return True, None
```

3) Validación con regex

Las regex son útiles para validar **formatos fijos**. Aquí solo trabajamos con patrones sencillos.

- [Documentación oficial Regex python](#)
- [Herramienta | Regex101](#) (tiene modo Python)
- [Herramienta | Regexpr](#)

3.1 `re.fullmatch`

Si quieres validar el string entero (no una parte), usa `fullmatch`:

```
import re
ok = re.fullmatch(PATRON, texto) is not None
```

Recomendaciones prácticas en Python:

- Usa **raw strings** (`r"..."`) para evitar escapes dobles con `\`.
- Para **validación**, prioriza `re.fullmatch` (exige que coincida todo el string).
- `re.match` coincide al **inicio** del string y `re.search` busca en **cualquier parte**.

3.2 Sintaxis en Python

```
import re

patron = r"ab+c"

es_valido = re.fullmatch(patron, "abbbc") is not None

m = re.search(patron, "xxx abbbc yyy")
```

```
if m:
    print(m.group(0))
```

Si vas a reutilizar el mismo patrón muchas veces:

```
import re

rx = re.compile(r"ab+c", flags=re.IGNORECASE)
rx.fullmatch("ABBC") is not None
```

3.3 Métodos importantes (re)

Equivalencias prácticas:

- `re.search(p, s)` → devuelve match o `None` (como "hay coincidencia")
- `re.match(p, s)` → match al inicio
- `re.fullmatch(p, s)` → match de todo el string (validación)
- `re.findall(p, s)` → lista de coincidencias
- `re.finditer(p, s)` → iterador de matches (útil si quieres posiciones)
- `re.sub(p, reemplazo, s)` → reemplazar
- `re.split(p, s)` → dividir

3.4 Flags (banderas) en Python

En Python se pasan como `flags=` y se combinan con `|`.

- `re.IGNORECASE` (o `re.I`): no distingue mayúsc/minúsc
- `re.MULTILINE` (o `re.M`): `^` y `$` actúan por línea
- `re.DOTALL` (o `re.S`): `.` incluye saltos de línea
- `re.VERBOSE` (o `re.X`): permite escribir regex con espacios y comentarios (muy útil para mantenerla)

Ejemplo:

```
rx = re.compile(r"^hola$", flags=re.I | re.M)
```

3.5 Mini-guía de "piezas" de regex (cheatsheet)

Comodines y clases

- `.` cualquier caracter (salvo salto de línea, a no ser que uses `re.S`)
- `[aeiou]` uno de esos caracteres
- `[a-z]` rango (ojo: por ASCII/Unicode, depende del caso)
- `\d` dígito (`[0-9]`)

- `\D` no dígito
- `\s` espacio en blanco (incluye tabs y saltos)
- `\S` no espacio en blanco
- `\w` alfanumérico + `_`
- `\W` no alfanumérico

Delimitadores y cantidad

- `^` inicio de string (o línea si `re.M`)
- `$` fin de string (o línea si `re.M`)
- `{n}` exactamente n
- `{n,m}` entre n y m
- `{n,}` al menos n
- `*` cero o más
- `+` uno o más
- `?` cero o uno

Alternativas

- `(a|b|c)` una de las opciones

Negación en clases

- `[^0-9]` cualquier cosa que **no** sea dígito

3.6 Ejemplos útiles en Python con Regex

3.6.1 Email

```
import re

EMAIL_RX = re.compile(r"^[\\w\\.\\-]+@[\\w\\.\\-]+\\.([A-Za-z]){2,}$")

def email_es_valido(email: str) -> bool:
    return EMAIL_RX.fullmatch(email.strip()) is not None
```

3.6.2 Teléfono 123-456-7890

```
PHONE_RX = re.compile(r"^\\d{3}-\\d{3}-\\d{4}$")
PHONE_RX.fullmatch("123-456-7890") is not None
```

3.6.3 Contraseña "fuerte" (ejemplo)

Esto ya es más "avanzado", pero sirve para mostrar lookaheads.

```
PWD_RX = re.compile(r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$")
PWD_RX.fullmatch("Passw0rd!") is not None
```

3.6.4 Extraer números de un texto

```
import re

text = "El precio es 50 y el descuento 10."
nums = re.findall(r"\d+", text)
nums # ['50', '10']
```

3.6.5 Reemplazar palabras

```
import re

sentence = "Python es genial. Me encanta Python."
new_sentence = re.sub(r"Python", "Rust", sentence)
```

3.6.6 Validar fecha DD/MM/YYYY con regex (formato)

Nota: esta regex valida el **formato** y rangos básicos, pero no todos los calendarios.

```
DATE_RX = re.compile(r"^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/\\d{4}$")
DATE_RX.fullmatch("07/10/2025") is not None
```

3.6.7 Nombre: solo letras y espacios (incluye tildes)

```
NAME_RX = re.compile(r"^[A-Za-z-ÁÉÍÓÚáéíóúÑñ\\s]+$")
NAME_RX.fullmatch("Juan Pérez") is not None
```

3.6.8 Validar Código postal (5 dígitos)

```
CP_RX = re.compile(r"^\d{5}$")
CP_RX.fullmatch("28001") is not None
```

3.6.9 Validar URL

Valida una URL "simple":

- `http://` o `https://` opcional
- `www.` opcional
- dominio con letras/números/puntos/guiones
- extensión de 2 o más letras

```
URL_RX = re.compile(r"^(https?://)?(www\\.?)?[A-Za-z0-9.-]+\.[A-Za-z]{2,}$")
URL_RX.fullmatch("https://www.example.com") is not None
```

3.6.10 Buscar palabras que empiecen por una letra (con `\\b`)

`\\b` marca un límite de palabra. Útil para “buscar palabras”.

```
text = "Hola, ¿cómo estás? Hoy es un buen día."
palabras = re.findall(r"\\bH[A-Za-zÁÉÍÓÚáéíóúÑñ]*\\b", text)
palabras # ['Hola', 'Hoy']
```